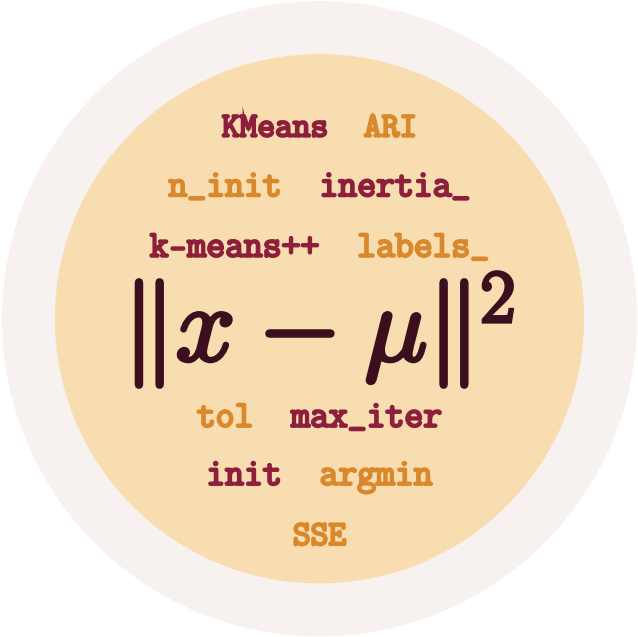
k-средних

Шпаргалка



Формулы и вызовы из урока «k-средних». Строка с формулой — что это, формула, ниже пример. Строка с кодом — что ищут, что делает, ниже пример и после стрелки его значение, снятое запуском. Примеры кода — на этих массивах:

```
import numpy as np
X = np.array([[1, 2], [2, 0], [4, 4], [5, 2]])
y = np.array([0, 0, 1, 1])
```

Алгоритм Ллойда

SSE — $\sum_i \|x_i - \mu_{c_i}\|^2$, c_i — кластер объекта
▷ точки 0, 2, 6, центры 1 и 6: SSE $= 1 + 1 + 0 = 2$

Шаг — назначение — к ближайшему центру, пересчёт — центр в среднее кластера; до неизменного назначения; для пустого кластера — своё правило
▷ точки 0, 2, 6, центры 0 и 6: кластеры $\{0, 2\}$ и $\{6\}$, центры 1 и 6

Среднее — лучший центр — $\sum_{i \in S} \|x_i - \mu\|^2 = \sum_{i \in S} \|x_i - \bar{x}\|^2 + |S| \cdot \|\bar{x} - \mu\|^2$
▷ точки 0, 2 и $\mu = 3$: $9 + 1 = 2 + 2 \cdot 4$

Останов — SSE не растёт ни на одном шаге; разбиений конечное число, но минимум бывает локальным — нужны разные старты
▷ 0, 1, 10, 11, 20, 21 со старта 0, 1, 11: SSE 101 против 1,5

k-means++ — первый центр — случайный объект, следующий — объект с вероятностью, пропорциональной квадрату расстояния до ближайшего центра
▷ расстояния 1, 2, 3: вероятности $\frac{1}{14}$, $\frac{4}{14}$, $\frac{9}{14}$

D — квадраты расстояний до центров — объектов 0 и 2

```
D = ((X[:, None] - X[[0, 2]]) ** 2).sum(axis=2)
```

`D.argmin(axis=1)` — назначение

```
D.argmin(axis=1) → [0 0 1 1]
```

`D.min(axis=1).sum()` — SSE назначения

```
D.min(axis=1).sum() → 10
```

Сравнить разбиения

Индекс Рэнда — доля пар объектов, на которых разбиения согласны: вместе в обоих или врозь в обоих

▷ $(0,0,1,1)$ и $(0,0,0,1)$: согласны 3 пары из 6, индекс $\frac{1}{2}$

ARI — $\dfrac{\sum_{ij}\binom{n_{ij}}{2} - E}{\frac{1}{2}\left(\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right) - E}$, $E = \dfrac{\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}}{\binom{n}{2}}$; 1 — совпадают с точностью до номеров, около 0 — не связаны

▷ те же разбиения: $\sum \binom{n_{ij}}{2} = 1$, $E = \frac{2\cdot 3}{6} = 1$, ARI $= 0$

Вызовы sklearn

Порядок — создать, `fit(X)`, здесь без ответов. Модули — `sklearn.cluster` и `sklearn.metrics`.

`KMeans` — `n_clusters`, `n_init`, `init`, `tol`, `random_state`

```
km = KMeans(2, n_init=10, random_state=0).fit(X)
```

`km.labels_` — номера кластеров, произвольные

```
km.labels_ → [1 1 0 0]
```

`km.cluster_centers_` — центры, строка на кластер

```
km.cluster_centers_.shape → (2, 2)
```

`km.inertia_` — SSE

```
km.inertia_ → 5.0
```

`adjusted_rand_score` — ARI двух разбиений, номера неважны

```
adjusted_rand_score(y, km.labels_) → 1.0
```

Где sklearn расходится с точными правилами

Старты — `n_init="auto"` — для k-means++ один запуск, его фиксирует `random_state`

▷ свой старт — `init=` массив центров и `n_init=1`

Останов — назначение не изменилось или сумма квадратов сдвигов центров не больше `tol` на среднюю дисперсию; не больше `max_iter`; после останова по `tol` центры не пересчитаны

▷ точный Ллойд — `tol=0`

Пустой кластер — центр переносится в объект, самый далёкий от своего центра

▷ в точном алгоритме — правило из условия